

베이지안 신경망

Bayesian neural networks

조성일

인하대학교
통계학과

Table of contents

1. Introduction
2. Artificial Neural Networks
3. Bayesian Neural Networks
4. Example

Introduction

Motivation I

- ▶ Neural networks (or deep neural networks) are usually trained using a (penalized) maximum likelihood objective to find a single setting of parameters.

Motivation I

- ▶ Neural networks (or deep neural networks) are usually trained using a (penalized) maximum likelihood objective to find a single setting of parameters.
- ▶ However, large flexible models like neural networks can represent many functions, corresponding to different parameter settings, which fit the training data well, yet generalize in different ways.

Motivation II

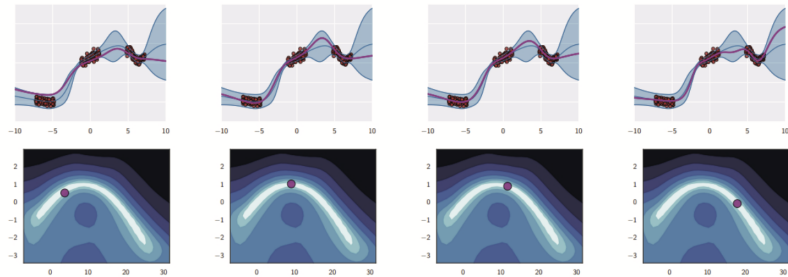


Figure: Muphy (2023). Figure 17.11 in Chapter 17

Motivation III

- ▶ Considering all of these different models together can lead to improved accuracy and uncertainty representation.

Motivation III

- ▶ Considering all of these different models together can lead to improved accuracy and uncertainty representation.
- ▶ This can be done by computing the posterior predictive distribution using Bayesian model averaging (BMA):

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \theta) p(\theta \mid \mathcal{D}) d\theta,$$

where $p(\theta \mid \mathcal{D}) \propto p(\theta) p(\mathcal{D} \mid \theta)$.

Artificial neural networks (ANNs)

ANNs: Introduction I

- ▶ Artificial neural networks (ANNs) are a class of computational models inspired by the structure of biological nervous systems.

ANNs: Introduction I

- ▶ Artificial neural networks (ANNs) are a class of computational models inspired by the structure of biological nervous systems.
- ▶ The basic building block of an ANN is a perceptron (or neuron), a simple computational unit that combines a linear transformation z of the input with a non-linear activation (or transfer) function $g(\cdot)$.

ANNs: Introduction I

- ▶ Artificial neural networks (ANNs) are a class of computational models inspired by the structure of biological nervous systems.
- ▶ The basic building block of an ANN is a perceptron (or neuron), a simple computational unit that combines a linear transformation z of the input with a non-linear activation (or transfer) function $g(\cdot)$.
- ▶ The output of the neuron is defined as:

$$\begin{aligned} z &= \sum_{i=1}^n w_i x_i + b, \\ y &= g(z) \end{aligned}$$

ANNs: Introduction II

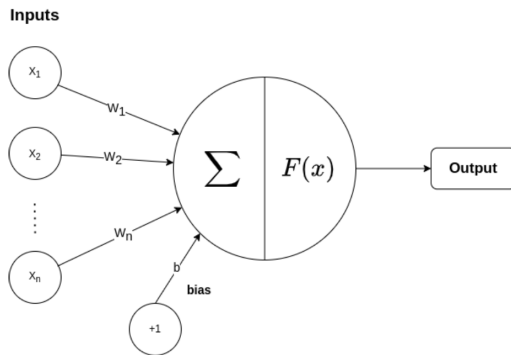


Figure: <https://bora.uib.no/bora-xmlui/bitstream/handle/11250/3101521/report.pdf?sequence=1&isAllowed=y>

ANNs: Feed Forward Neural Networks I

- ▶ Feedforward neural networks (FFNNs), aka multi-layer perceptrons (MLPs), are the most basic type of ANNs.

ANNs: Feed Forward Neural Networks I

- ▶ Feedforward neural networks (FFNNs), aka multi-layer perceptrons (MLPs), are the most basic type of ANNs.
- ▶ It is known that an MLP is a universal approximator, meaning it can model any suitably smooth function to any desired level of accuracy (Hornik 1992).

ANNs: Feed Forward Neural Networks I

- ▶ Feedforward neural networks (FFNNs), aka multi-layer perceptrons (MLPs), are the most basic type of ANNs.
- ▶ It is known that an MLP is a universal approximator, meaning it can model any suitably smooth function to any desired level of accuracy (Hornik 1992).
- ▶ MLPs consist of multiple neurons organized into layers:
1) input layer, 2) hidden layers, 3) output layer.

ANNs: Feed Forward Neural Networks II

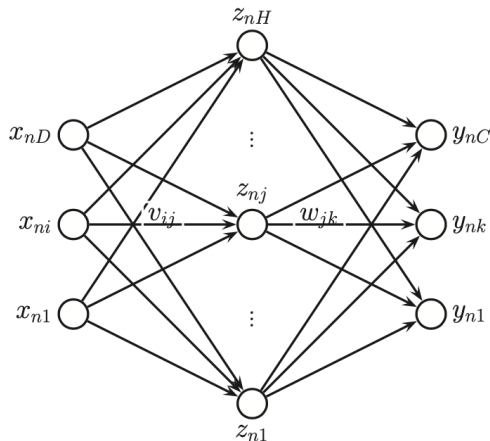


Figure: Murphy (2012). Figure 16.11 in Chapter 16

ANNs: Activation functions

- ▶ Activation functions are a crucial part of the design of ANNs.

ANNs: Activation functions

- ▶ Activation functions are a crucial part of the design of ANNs.
- ▶ The activation functions determine the transformation of the weighted sum of the inputs into the output of the neuron.

ANNs: Activation functions

- ▶ Activation functions are a crucial part of the design of ANNs.
- ▶ The activation functions determine the transformation of the weighted sum of the inputs into the output of the neuron.
- ▶ Commonly used activation functions are
 - ▶ Sigmoid
 - ▶ ReLU (Rectified Linear Unit)

ANNs: Activation functions - Sigmoid

- ▶ The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

ANNs: Activation functions - Sigmoid

- ▶ The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- ▶ The sigmoid function is usually utilized for binary classification problem.

ANNs: Activation functions - Sigmoid

- ▶ The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- ▶ The sigmoid function is usually utilized for binary classification problem.
- ▶ The gradient of the sigmoid function is small for values that are far from zero, which is known as being able to lead to the problem of **vanishing gradients** during training.

ANNs: Activation functions - ReLU

- ▶ The rectified linear unit (ReLU) is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

ANNs: Activation functions - ReLU

- ▶ The rectified linear unit (ReLU) is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

- ▶ The ReLU function has been demonstrated to improve training in deep neural networks (Agarap, 2019).

ANNs: Activation functions - ReLU

- ▶ The rectified linear unit (ReLU) is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

- ▶ The ReLU function has been demonstrated to improve training in deep neural networks (Agarap, 2019).
- ▶ The ReLU function is also less prone to the problem of **vanishing gradients** since it has a constant gradient for values greater than zero.

Bayesian neural networks (BNNs)

BNNs: Introduction I

- The main challenges in applying Bayesian inference to ANNs (or DNNs) are specifying suitable priors, and efficiently computing the posterior.

BNNs: Introduction I

- ▶ The main challenges in applying Bayesian inference to ANNs (or DNNs) are specifying suitable priors, and efficiently computing the posterior.
- ▶ Particularly, the posterior computation is more challenging due to the large number of parameters and the large datasets.

BNNs: Introduction I

- ▶ The main challenges in applying Bayesian inference to ANNs (or DNNs) are specifying suitable priors, and efficiently computing the posterior.
- ▶ Particularly, the posterior computation is more challenging due to the large number of parameters and the large datasets.
- ▶ The application of Bayesian inference to DNNs is sometimes called **Bayesian deep learning (BDL)**.

BNNs: Introduction II

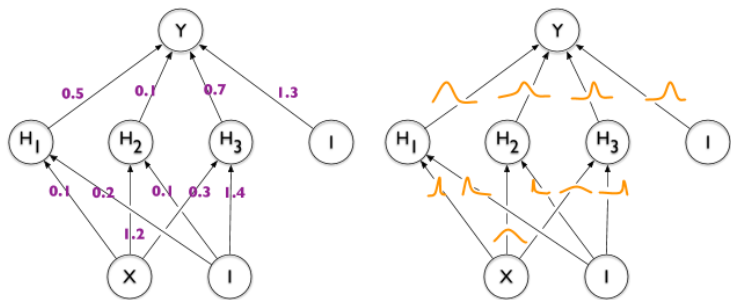


Figure: <https://sanjaykthakur.com/2018/12/05/the-very-basics-of-bayesian-neural-networks/>

BNNs: Gaussian prior I

- Consider an MLP with $L - 1$ hidden layers and a linear output

$$f(x; \theta) = W_L(\cdots g(W_2 g(W_1 x + b_1) + b_2)) + b_L,$$

where $\theta = (W_1, \dots, W_L, b_1, \dots, b_L)$.

BNNs: Gaussian prior I

- Consider an MLP with $L - 1$ hidden layers and a linear output

$$f(x; \theta) = W_L(\cdots g(W_2 g(W_1 x + b_1) + b_2)) + b_L,$$

where $\theta = (W_1, \dots, W_L, b_1, \dots, b_L)$.

- The most popular prior is a factorized Gaussian prior

$$W_l \sim \mathcal{N}(0, \alpha_l^2 I), \quad b_l \sim \mathcal{N}(0, \beta_l^2 I).$$

BNNs: Gaussian prior II

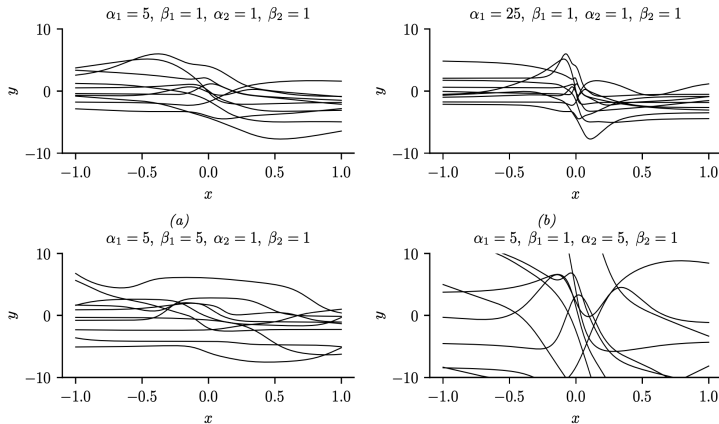


Figure: Murphy (2023). Figure 17.1 in Chapter 17.

BNNs: Gaussian prior III

- ▶ Setting hyperparameters:
 - ▶ Xavier initialization or Glorot initialization

$$\alpha_l^2 = \frac{2}{n_{in} + n_{out}}, l = 1, \dots, L.$$

BNNs: Gaussian prior III

- ▶ Setting hyperparameters:

- ▶ Xavier initialization or Glorot initialization

$$\alpha_l^2 = \frac{2}{n_{in} + n_{out}}, l = 1, \dots, L.$$

- ▶ LeCun initialization

$$\alpha_l^2 = \frac{1}{n_{in}}, l = 1, \dots, L.$$

Example

Regression

- ▶ Regression model

$$y_i = f(x_i) + \epsilon_i, \quad i = 1, \dots, n,$$

where ϵ_i are errors following normal distribution with mean zero and variance σ^2 . That is,

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Regression: BNN I

- ▶ activation function

```
# the non-linearity we use in our neural network
def nonlin(x):
    return jax.nn.tanh(x)
```

Regression: BNN II

► BNN - model

```
# a two-layer bayesian neural network with computational flow
# given by  $D_X \Rightarrow D_H \Rightarrow D_Y$  where  $D_H$  is the number of
# hidden units. (note we indicate tensor dimensions in the comments)
def model(X, Y, D_H, D_Y=1):
    N, D_X = X.shape

    # sample first layer (we put unit normal priors on all weights)
    w1 = numpyro.sample("w1", dist.Normal(jnp.zeros((D_X, D_H)), jnp.ones((D_X, D_H))))
    assert w1.shape == (D_X, D_H)
    z1 = nonlin(jnp.matmul(X, w1)) # <= first layer of activations
    assert z1.shape == (N, D_H)

    # sample second layer
    w2 = numpyro.sample("w2", dist.Normal(jnp.zeros((D_H, D_H)), jnp.ones((D_H, D_H))))
    assert w2.shape == (D_H, D_H)
    z2 = nonlin(jnp.matmul(z1, w2)) # <= second layer of activations
    assert z2.shape == (N, D_H)

    # sample final layer of weights and neural network output
    w3 = numpyro.sample("w3", dist.Normal(jnp.zeros((D_H, D_Y)), jnp.ones((D_H, D_Y))))
    assert w3.shape == (D_H, D_Y)
    z3 = jnp.matmul(z2, w3) # <= output of the neural network
    assert z3.shape == (N, D_Y)

    if Y is not None:
        assert z3.shape == Y.shape

    # we put a prior on the observation noise
    prec_obs = numpyro.sample("prec_obs", dist.Gamma(3.0, 1.0))
    sigma_obs = 1.0 / jnp.sqrt(prec_obs)

    # observe data
    with numpyro.plate("data", N):
        # note we use to_event(1) because each observation has shape (1,)
        numpyro.sample("Y", dist.Normal(z3, sigma_obs).to_event(1), obs=Y)
```


Regression: variational distribution

- ▶ variational distribution

$$q(\theta) = \mathcal{N}(\theta \mid \mu, \Sigma)$$

```
from numpyro.infer.autoguide import AutoMultivariateNormal
from numpyro.infer.autoguide import AutoDiagonalNormal

guide = AutoMultivariateNormal(model)
```

Remark and next week

Remark

- ▶ Artificial neural networks can represent many functions, corresponding to different parameter settings, which fit the training data well, yet generalize in different ways.
- ▶ In these cases, Bayesian neural networks can lead to improved accuracy and uncertainty representation.

Next week

► Gaussian processes

References

- ▶ NumPyro tutorials
- ▶ Murphy (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
- ▶ Murphy (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press.